



ThreatLens — Command Reference

Threat Detection & URL Scanner · FastAPI + SQLite + Vanilla JS dashboard

Python 3.11+

FastAPI · Uvicorn

SQLite

Works with zero API keys

1 Quick Start (already set up)

Your environment is ready: `backend/.venv` exists with all dependencies and the database is created automatically on first run.

```
# From the project root – Git Bash on Windows
cd backend
source .venv/Scripts/activate
uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload
```

2 One-Liner (no venv activation)

```
cd backend
.venv/Scripts/python.exe -m uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload
```

3 Fresh Setup From Scratch

```
# 1. Create & activate a virtual environment
cd backend
python -m venv .venv
source .venv/Scripts/activate      # PowerShell: .venv\Scripts\Activate.ps1

# 2. Install dependencies
pip install -r requirements.txt

# 3. (Optional) configure threat-intel keys
cp .env.example .env

# 4. Run the server
uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload
```

4 Optional: Threat-Intel API Keys

ThreatLens works with **zero API keys**. Add any of the following to `backend/.env` to enrich scan results (all providers degrade gracefully):

```
VIRUSTOTAL_API_KEY=your_key_here
GOOGLE_SAFE_BROWSING_API_KEY=your_key_here
PHISHTANK_API_KEY=your_key_here
ADMIN_USERNAME=alex           # auto-promotes this user to admin on startup
```

The frontend is served automatically by the backend at `/` — there is **no separate frontend command**.

5 Where to Look After Starting

URL	What it is
<code>http://localhost:8000</code>	Dashboard — register an account, then use the scanner, phishing lab, history, intel & profile pages
<code>http://localhost:8000/api/docs</code>	Interactive API docs (Swagger) — add a bearer token via <i>Authorize</i>
<code>http://localhost:8000/api/health</code>	Service health check → <code>{"status": "ok", ... }</code>

Built-in admin account

Username	Password	Role
<code>admin024</code>	<code>admin2412</code>	Admin — redirected to the admin panel after login

6 API Examples (curl)

Register & login

```
curl -X POST http://localhost:8000/api/auth/register \  
  -H "Content-Type: application/json" \  
  -d '{"username": "alex", "email": "alex@mail.com", "password": "ThreatLens!2026"}'  
  
curl -X POST http://localhost:8000/api/auth/login \  
  -H "Content-Type: application/json" \  
  -d '{"identifier": "alex", "password": "ThreatLens!2026"}'
```

Scan a URL (uses the token from login)

```
curl -X POST http://localhost:8000/api/scan-url \  
  -H "Authorization: Bearer <token>" \  
  -H "Content-Type: application/json" \  
  -d '{"url": "http://paypal-account-verify.tk/login"}'  
  
# Analyze phishing content  
curl -X POST http://localhost:8000/api/analyze-phishing \  
  -H "Authorization: Bearer <token>" \  
  -H "Content-Type: application/json" \  
  -d '{"content_type": "email", "content": "URGENT: your account will be closed..."}'
```

7 Other Useful Commands

Test suite

```
cd backend  
.venv/Scripts/python.exe test_e2e.py
```

Stop / restart

```
Ctrl+C          # stop the running server in the terminal  
uvicorn app.main:app --reload # start again (change is picked up automatically)
```

Docker (optional deployment)

```
docker build -t threatlens .  
docker run -p 8000:8000 threatlens
```

8 Production Notes

- **Workers:** `uvicorn app.main:app --workers 4`
- **CORS:** tighten `allow_origins=["*"]` in `backend/app/main.py`
- **Database:** swap to Postgres via `DATABASE_URL` in `.env` — no code changes needed
- **Run behind a reverse proxy** (nginx/Caddy) with TLS termination